calls
<<route>>
ConversationRoutes
+GET /conversations (AuthGuard, PermissionGuard)
+POST /conversations (AuthGuard, PermissionGuard)
+GET /conversations/:id/messages (AuthGuard, PermissionGuard)
+POST /conversations/:id/messages (AuthGuard, PermissionGuard)
+POST /conversations/:id/files (AuthGuard, PermissionGuard)
+DELETE /conversations/:id/messages/:messageId (AuthGuard, PermissionGuard)
+PATCH /conversations/:id/read (AuthGuard, PermissionGuard)
+PATCH /conversations/:id/hide (AuthGuard, PermissionGuard)
+PATCH /conversations/:id/pin (AuthGuard, PermissionGuard)
maps to
ConversationController
-conversationService: ConversationService
+getConversations(userId)
+createConversation(userId, body)
+getMessages(userId, id)
+sendMessage(userId, id, body)
+uploadFile(userId, id, file)
+deleteMessage(userId, id, messageId)
+markAsRead(userId, id)
+hideConversation(userId, id)
+pinConversation(userId, id, pinned)
AuthGuard
-tokenService: TokenService
-prisma: PrismaService
-reflector: Reflector
+canActivate(context: ExecutionContext): Promise<boolean>
-extractTokenFromHeader(request: Request): string | undefined
PermissionGuard
-reflector: Reflector
-sharedPermissionRepository: SharedPermissionRepository
+canActivate(context: ExecutionContext): Promise<boolean>
<<dto>>
CreateConversationBodyDto
+participantId: number
<<dto>>
SendMessageBodyDto
+message: string (max 2000)
+attachment?: MessageAttachmentType
<<dto>>
PinConversationBodyDto
+pinned: boolean
<<type>>
ConversationListItemType
+otherUser: { id, profile }
+lastMessage: DirectMessageType | null
+unreadCount: number
+pinnedAt: Date | null
<<type>>
MessageAttachmentType
+fileUrl: string
+fileName: string
+fileSize: number | null
+fileType: string | null
ConversationService
-conversationRepository: ConversationRepository
+getConversations(userId)
+getConversationItem(conversationId, userId)
+getMessages(userId, conversationId)
+createConversation(userId, participantId)
+sendMessage(userId, id, message, attachment?)
+uploadFile(userId, id, file)
+deleteMessage(userId, id, messageId)
+markAsRead(userId, id)
+hideConversation(userId, id)
+pinConversation(userId, id, pinned)
+ensureParticipant(conversationId, userId)
-buildConversationItem(conversation, userId)
calls
ConversationRepository
-prisma: PrismaService
+findConversationBetween(userA, userB)
+getConversationsByUserId(userId)
+findConversationByIdForUser(id, userId)
+findConversationById(id)
+hideConversationForUser(id, userId)
+unhideConversation(id)
+setPinnedForUser(id, userId, pinned)
+getMessages(conversationId)
+createConversation(userA, userB)
+createMessage(conversationId, senderId, body)
+softDeleteMessage(conversationId, messageId, userId)
+markMessagesAsRead(conversationId, userId)
+countUnread(conversationId, userId)
maps to
maps to
<<schema>>
Conversation
+id: number
+participant1: number
+participant2: number
+createdAt: Date
+deletedAt: Date | null
+participant1HiddenAt: Date | null
+participant2HiddenAt: Date | null
+participant1PinnedAt: Date | null
+participant2PinnedAt: Date | null
1..*
<<schema>>
DirectMessage
+id: number
+conversationId: number
+senderId: number
+message: string
+fileUrl: string | null
+fileName: string | null
+fileSize: number | null
+fileType: string | null
+isRead: boolean
+createdAt: Date
+deletedAt: Date | null

calls
<<route>>
ConversationSocketEvents
+connect (JWT handshake)
+disconnect
+message:send
+message:read
+message:delete
+conversation:hide
+typing
maps to
ConversationsGateway
-conversationService: ConversationService
-tokenService: TokenService
+handleConnection(client: Socket)
+handleDisconnect(client: Socket)
+handleSendMessage(payload)
+handleMarkAsRead(payload)
+handleDeleteMessage(payload)
+handleHideConversation(payload)
+handleTyping(payload)
<<type>>
SendMessagePayload
+conversationId: number
+message: string (max 2000)
+attachment?: MessageAttachmentType
<<type>>
MarkAsReadPayload
+conversationId: number
<<type>>
HideConversationPayload
+conversationId: number
<<type>>
DeleteMessagePayload
+conversationId: number
+messageId: number
<<type>>
TypingPayload
+conversationId: number
+isTyping: boolean
ConversationService
-conversationRepository: ConversationRepository
+sendMessage(userId, id, message, attachment?)
+markAsRead(userId, id)
+deleteMessage(userId, id, messageId)
+hideConversation(userId, id)
+getConversationItem(conversationId, userId)
+ensureParticipant(conversationId, userId)
calls
ConversationRepository
-prisma: PrismaService
+createMessage(conversationId, senderId, body)
+unhideConversation(id)
+markMessagesAsRead(conversationId, userId)
+softDeleteMessage(conversationId, messageId, userId)
+hideConversationForUser(id, userId)
+countUnread(conversationId, userId)
maps to
maps to
<<schema>>
Conversation
+id: number
+participant1: number
+participant2: number
+createdAt: Date
+deletedAt: Date | null
+participant1HiddenAt: Date | null
+participant2HiddenAt: Date | null
+participant1PinnedAt: Date | null
+participant2PinnedAt: Date | null
1..*
<<schema>>
DirectMessage
+id: number
+conversationId: number
+senderId: number
+message: string
+fileUrl: string | null
+fileName: string | null
+fileSize: number | null
+fileType: string | null
+isRead: boolean
+createdAt: Date
+deletedAt: Date | null